Software Architecture and Design Specification

Traffic Management System (Simulation)

Document control	Value
Course	Software Engineering UE24CS341A
Institution	PESU
Document edition	1.1 - consolidated to Baseline Amendment v1.1
Normative baseline	SAD edition 1.1; requirements follow SRS normative baseline 1.0
Revision date	6 November 2026
Authors	Venkata Srivaibhav Nukaraju; V Vyas; Vikas N Naik; Vikhyath Bharadwaj K S
Status	Consolidated for submission; approval claims limited to recorded team decision

Revision history

Edition	Date	Change	Approval
1.0	2026-09-14	Original Software Architecture and Design Specification; source PDF preserved.	Pending in original record
1.1	6 November 2026	Consolidated edition incorporating Baseline Amendment v1.1; does not claim new faculty approval.	No new approval claimed

Course Coordinator approval is not recorded in the source documents reviewed.

1 Introduction

1.1 Purpose and scope

This document describes the architecture of TMS-Sim, a local microscopic traffic-simulation workbench. Scope includes network and signal editing, demand generation, simulation, 2D visualization, metrics/results, local scenario/run storage and CSV export. Real-world signal/sensor hardware, 3D, distributed execution, advanced lane-changing, pedestrians, cyclists and public-transport simulation are out of scope for v1.

1.2 Audience and references

Readers include the project team, Project Guide, evaluators, QA/testers, Analysts and future maintainers. Related records: SRS_v1.1, docs/rtm_draft.csv, docs/test_catalogue.md, Baseline Amendment v1.1, Decision Record 0001 and DEVIATIONS.md.

1.3 Terms

TMS-Sim: Traffic Management System (Simulation); SRS/STP: Software Requirements Specification/Software Test Plan; RTM: Requirements Traceability Matrix; MOE: Measure of Effectiveness; PRNG/seed: pseudo-random generator/initial state; FPS: frames per second; car-following and gap acceptance: documented vehicle movement rules.

2 Goals, constraints and stakeholders

- Provide a clear desktop workbench for traffic Analysts.

- Support reproducible microscopic simulation and validate network/signal inputs before execution.

- Visualize traffic in 2D and calculate comparable measures.

- Separate presentation, simulation/business logic and data/reporting responsibilities for maintainability and testing.

- Windows 10/11; Python 3.11 + PySide6; one local machine; no required network communication.

- Fixed-time signals and documented car-following/gap-acceptance model; no distributed system or 3D.

Stakeholder	Concern
Traffic Analyst	Understandable workflow, validation messages, controls, visualization and results.
Development team	Modularity, maintainability and testability.
Project Guide / evaluators	Correctness, traceability and acceptance evidence.
QA / future maintainers	Reproducibility, measurable tests and understandable components.

3 Architecture

3.1 Pattern and component view

A layered desktop architecture separates presentation, simulation/business logic and data/reporting. A distributed or microservice architecture is not appropriate for the single-machine, no-network scope.

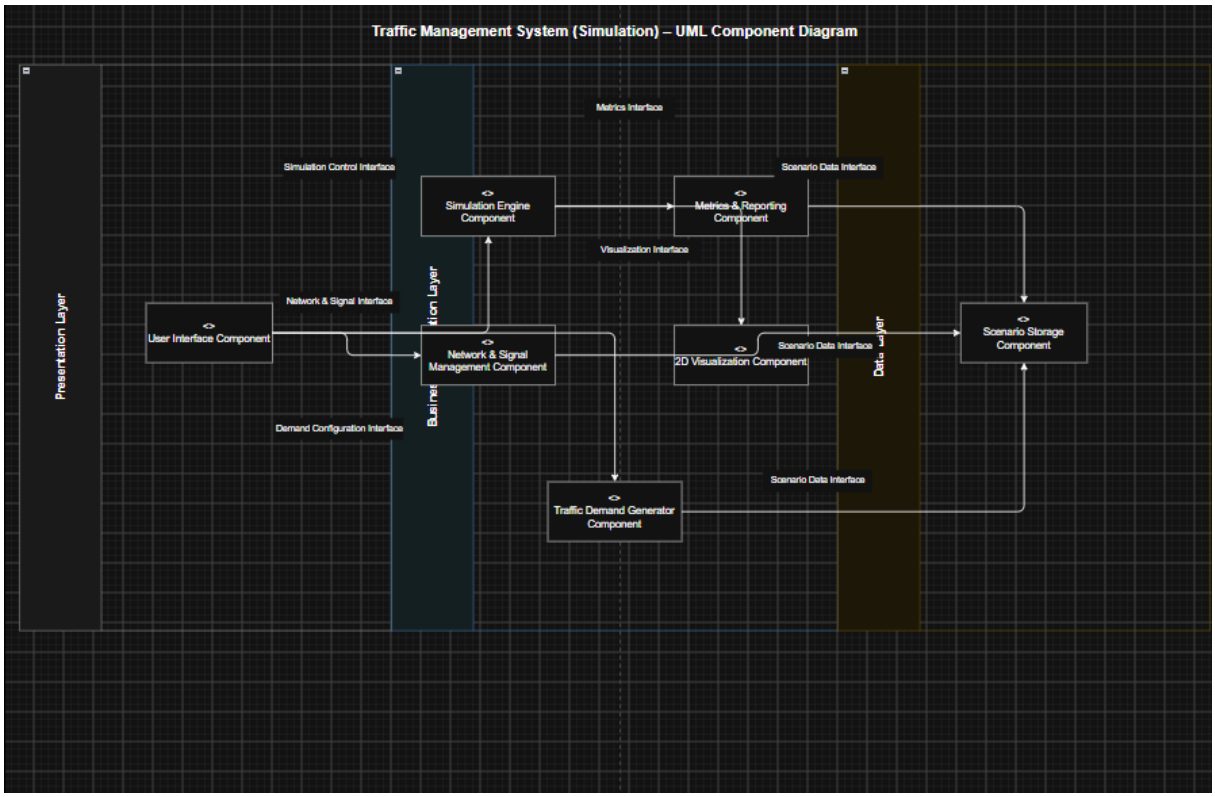


Figure 1. Original UML component diagram, retained from the Part 1 SAD.

Component	Responsibilities and contracts
Workbench / presentation	Coordinate editing, validation, controls, results views and local library workflows.
Network editor / NetworkService	Edit intersections, segments, lanes and turns; validateNetwork(), loadNetwork(), provideNetworkState().
Signal Configuration / SignalService	Define fixed-time phases; validateSignalPlan(plan, network), provideSignalPlan().
Demand Generator	generate_vehicles(seed, rates, ratios, *, duration_s) returns a bounded reproducible arrival stream.
Simulation Engine	Advance fixed steps and expose start/pause/step/reset/set_speed(0.5|1|2|4)/advance. Lifecycle events and completion callback on_finish(engine) are available; callback fires once per run.
2D Renderer	Display vehicle positions, signals and queues; provide pan/zoom.
Metrics / Results	Compute delay, per-approach queue, throughput, average speed/travel time; display live/final values and compare completed runs.
ScenarioStore / Report Exporter	Persist local scenarios and completed runs; exchange versioned JSON; export CSV summaries.

Run flow: Analyst to Workbench to Network/Signal validation to Demand Generator to Engine to Metrics to ScenarioStore/Results. Compare/export: Analyst to Results Dashboard to Metrics/ScenarioStore to Report Exporter.

3.2 Technology and data stores

- Python 3.11 and PySide6 desktop application. The baseline identifies no additional runtime dependency beyond PySide6 and Python standard library.

- SQLite via stdlib sqlite3 for the local scenario library and completed runs.

- Version-tagged JSON schema_version 1.0 for scenario import/export; CSV for completed-run summaries.

- No internet required for ordinary modelling and simulation; data remain on one local machine.

3.3 Security architecture

Access control is deferred. V1 is a local, single-Analyst application without accounts, roles, administration or audit log. Boundary checks validate scenario schema and version before data is used. Security is limited to safe local input and storage, not authentication or authorization.

3.4 Risks and traceability

Risk	Mitigation
Invalid/disconnected network or signal timing	Validate before run; block invalid/unsafe execution and locate issues.
Non-reproducible demand	Use one seedable PRNG and deterministic ordering.
Visualization slowdown	Measure the reference scenario on reference hardware against >=15 full-hour average FPS.
Malformed or wrong-version local scenario	Validate schema/version before use; exercise import and storage error cases.

Requirement-to-component mapping: TMS-F-010..013 to Network Editor/Store; F-020/F-022 to Signal Configuration; F-030/F-032 to Demand Generator; F-040..043 to Engine/Renderer; F-050..054 to Metrics/Results/Exporter; F-060 to Store; NF-001..004 to cross-cutting performance/reliability/usability/portability. Detailed test/status mapping is in docs/rtm_draft.csv.

4 Design

4.1 Interaction sequences

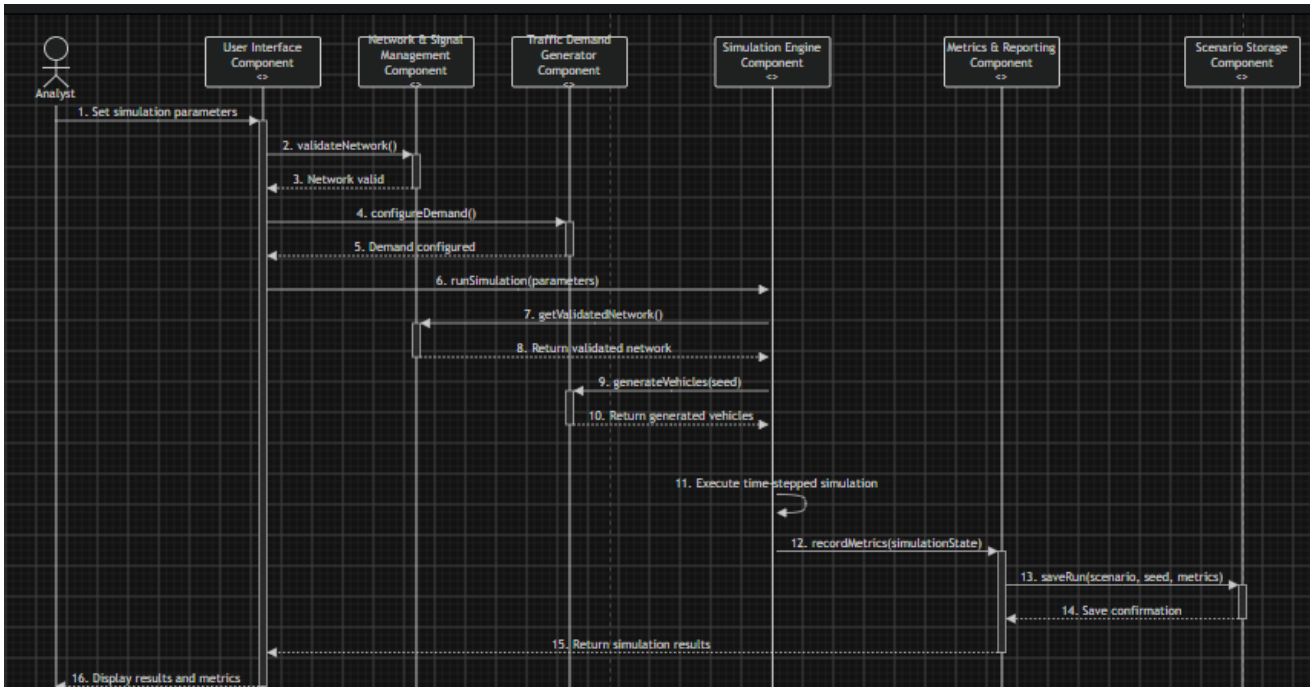


Figure 2. Original Run Simulation sequence, retained from the Part 1 SAD.

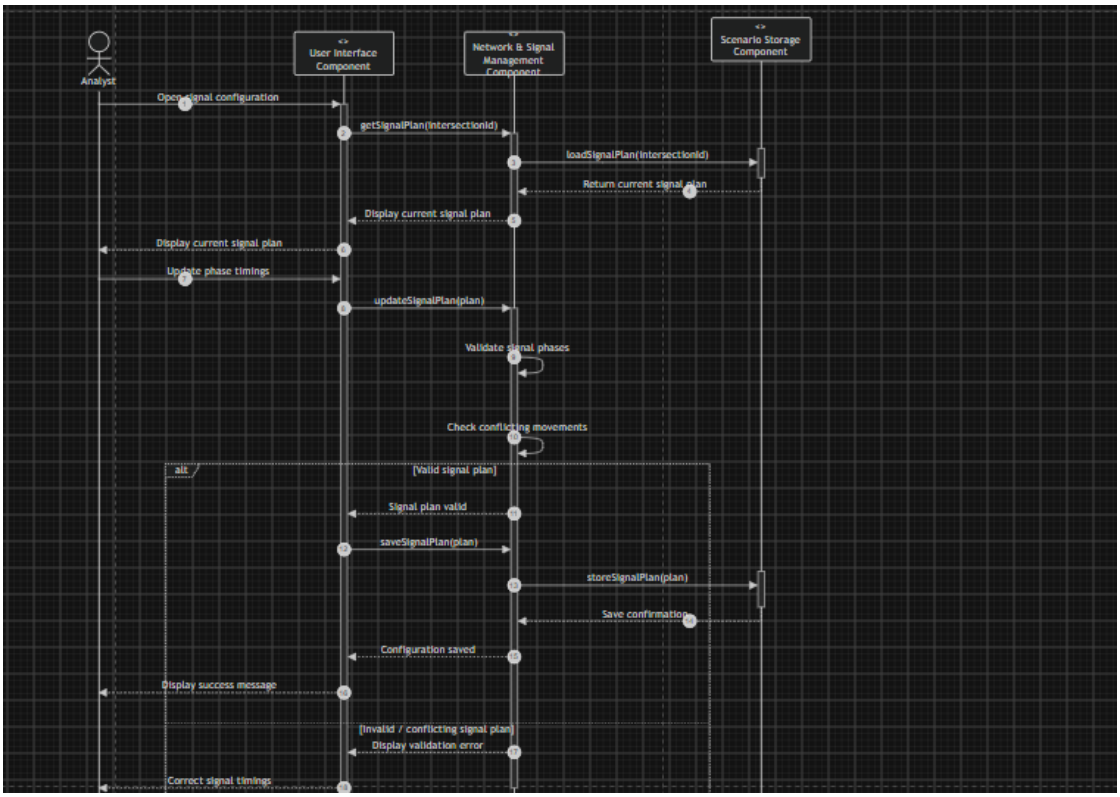


Figure 3. Original Configure Traffic Signal Plan sequence, retained from the Part 1 SAD.

Flow	Sequence
Run simulation	Configure; validate network and signal plan; create seeded arrivals; run Engine; calculate metrics; persist completed run; show results.
Configure signal	Edit phase plan; validate plan against network; expose valid plan to Engine; report located issues and block invalid plan.
Compare and export	Select two persisted runs; show metric values/differences and compatibility information; write CSV summary.

4.2 Internal interfaces

Interface	Contract
NetworkService	validateNetwork(), loadNetwork(), provideNetworkState().
SignalService	validateSignalPlan(plan, network), provideSignalPlan().
Demand to Engine	generate_vehicles(seed, rates, ratios, *, duration_s).
Engine to Metrics	Lifecycle tuple (time, vehicle_id, spawn|exit, segment_id); engine.exited contains completed trips; on_finish(engine) fires once per run.
Engine controls	start(), pause(), step(), reset(), set_speed(0.5|1|2|4), advance(wall_seconds).
ScenarioStore between Workbench	save_scenario, list_scenarios, load_scenario, duplicate_scenario, rename_scenario, delete_scenario, import_scenario, export_scenario, save_run, list_runs, load_run. Amendment aliases: saveScenario/loadScenario/listScenarios/saveRun.

Validation results contain issues with element id, element type, code, message and severity (error/warning). Valid means zero errors; warnings do not block a run. Signal validation accepts the network.

4.3 Model and safety constraints

IDM parameters: v0 equals segment speed limit; a_max 1.5 m/s2; b 2.0 m/s2; s0 2.0 m; T 1.5 s; braking floor 9 m/s2; vehicle length 5.0 m; fixed time step and trapezoidal position update. Only green is green; yellow/all-red are red, except a physically unavoidable yellow clearance. Stop/yield control uses a 3.0 s critical gap; stationary conflicting vehicles are ignored to avoid deadlock. Stop-controlled approaches are treated as yield in v1. Crossing or merging paths conflict; same-approach exception applies only to distinct receiving segments; different approaches into one receiving segment conflict. Missing position is a located validation error. Left-hand traffic, y-up model coordinates.

4.4 Error handling, monitoring and UX

Show clear validation issues linked to the affected element where possible; block execution on invalid network/signal input; use standardized simulation/result errors. Acceptance evidence covers completion, visualization FPS, reproducibility and metric correctness. The workbench uses central canvas, inspector, toolbox, controls and dockable results dashboard with live charts and post-run summaries.

4.5 Baseline status and design work

Baseline Amendment v1.1 finalizes the component interfaces. The original UML component and sequence figures are retained from the Part 1 SAD; the updated text and contracts record the approved changes. Changes outside approved v1 scope require baseline review. Code differences remain in DEVIATIONS.md.

Appendix A References

- SRS_v1.1 and docs/rtm_draft.csv.

- Baseline Amendment v1.1 and Decision Record 0001.

- IEEE Std 830-1998; UE24CS341A requirements engineering material; Treiber, Hennecke and Helbing (2000), Intelligent Driver Model.

- Implementation tools: Python 3.11, PySide6, sqlite3, JSON, CSV; draw.io or PlantUML for UML.